



Systemes paralleles et distribues

Projet : Parallel Ant Foraging

Javier andres TARAZONA JIMENEZ

Jose Daniel CHACON

Encadrant : JUVIGNY Xavier

Table des matières

1	Introduction	2
1.1	Objectifs de la simulation	2
2	Analyse du problème - Baseline	3
2.1	État de l'art	3
2.2	Profilage et identification des goulots d'étranglement	3
2.3	Résultats	4
2.3.1	Analyse des Résultats	5
3	Vectorisation	7
3.1	Stratégie de Vectorisation	7
3.1.1	Motivation	7
3.1.2	Du layout AoS au layout SoA	7
3.1.3	Maintien du baseline : un binaire, deux layouts	7
3.1.4	Choix d'implémentation : types compacts et seed par fourni	7
3.1.5	Application dans le noyau : <code>advance_time_soa</code>	8
3.1.6	Découplage de <code>SDL_Point</code> hors hot loop et adaptation du rendu	8
3.1.7	Validation et protocole de comparaison	8

1 Introduction

1.1 Objectifs de la simulation

L'objectif principal de la simulation est de modéliser le problème de fourragement d'une colonie de fourmis artificielles sur un terrain 2D hétérogène, afin d'identifier des trajets efficaces entre la fourmilière et une source de nourriture. Le cadre retenu suit un modèle d'intelligence en essaim où des agents simples produisent un comportement collectif émergent à partir de règles locales.

Le modèle vise à reproduire une coordination indirecte entre agents via deux types de phéromones : un phéromone d'exploration vers la nourriture (V_1) et un phéromone de retour vers le nid (V_2). Chaque fourmi évolue avec deux états (*unloaded*/*loaded*) et applique une politique exploration–exploitation pilotée par le paramètre ε , avec des transitions d'état lors de l'arrivée sur la nourriture ou sur la fourmilière.

L'environnement a pour objectif de représenter une difficulté de déplacement réaliste : chaque cellule de la carte possède un coût de traversée dans $[0, 1]$, et les zones non franchissables sont traitées comme obstacles. Le paysage est généré de manière fractale afin d'introduire des gradients de difficulté non uniformes et de contraindre la dynamique des déplacements.

Enfin, la simulation poursuit un objectif d'évaluation fonctionnelle au niveau de la colonie : la quantité de nourriture rapportée au nid sert d'indicateur global de performance. Ce cadre constitue la base expérimentale du projet, avant les étapes d'optimisation algorithmique (vectorisation et parallélisation) décrites dans le contexte général.

2 Analyse du problème - Baseline

2.1 État de l'art

L'implémentation actuelle initialise d'abord l'environnement dans `fractal_land.cpp` en construisant une carte fractale par raffinement récursif de sous-grilles. Dans `ant_simu.cpp`, cette carte est ensuite normalisée pour ramener les altitudes dans l'intervalle $[0, 1]$, ce qui fixe le coût de déplacement utilisé par les fourmis pendant la simulation.

La population est ensuite instanciée avec un nombre fixe de fourmis, des positions initiales pseudo-aléatoires et des paramètres globaux du modèle (*exploration*, *bruit*, *évaporation*). La structure `pheromone` (dans `pheromone.hpp`) maintient deux valeurs de phéromones par cellule, un tampon de mise à jour, ainsi que des bordures fantômes marquées à -1 pour empêcher les traversées hors domaine.

À chaque itération, la boucle principale de `ant_simu.cpp` traite les événements SDL puis exécute `advance_time`. Cette routine appelle `ant::advance` pour chaque agent, applique l'évaporation globale des phéromones (`do_evaporation`) puis valide l'état suivant de la carte (`update`). L'évolution individuelle d'une fourmi, définie dans `ant.cpp`, combine un choix local de voisin (aléatoire ou guidé par le maximum de phéromone), l'accumulation d'un budget de mouvement (`consumed_time < 1`), le marquage local des phéromones et la mise à jour de l'état chargé/non chargé.

Le rendu de l'état simulé est assuré par `render.cpp` et `window.cpp`. Le rendu affiche le terrain, la position des fourmis, la distribution des deux phéromones et la courbe de nourriture cumulée. La couche fenêtre/rendering SDL est construite avec mécanismes de repli (*fallback*) pour conserver l'exécution même lorsque certaines options matérielles ne sont pas disponibles.

2.2 Profilage et identification des goulots d'étranglement

Le profilage a pour objectif d'isoler les coûts dominants de la simulation avant toute optimisation, en particulier avant les étapes de vectorisation et de parallélisation. Nous présentons ici la définition des blocs à mesurer, instrumentation temporelle, protocole d'exécution reproductible et structure des résultats attendus. Cette section décrit donc la méthode de profilage et les objets techniques mobilisés, sans présenter de résultats chiffrés.

TABLE 1 – Découpage des parties mesurées pour le profilage de la simulation

ID	Nature	Rôle	Périmètre
P0	Initialisation	Génération fractale du terrain	One-shot (démarrage)
P1	Initialisation	Normalisation des coûts du terrain	One-shot (démarrage)
P2	Initialisation	Création et placement de la population	One-shot (démarrage)
K0	Noyau simulation	Coût total du noyau par itération	Itératif
K1	Noyau simulation	Avance globale de toutes les fourmis	Itératif
K2	Noyau simulation	Logique interne hors marquage phéromones	Itératif
K3	Noyau simulation	Marquage local des phéromones	Itératif
K4	Noyau simulation	Évaporation globale des phéromones	Itératif
K5	Noyau simulation	Validation/swap de la carte de phéromones	Itératif
R0	Rendu SDL	Affichage (terrain, fourmis, phéromones, courbe)	Itératif, hors noyau
E0	Événements SDL	Traitement de <code>SDL_PollEvent</code>	Itératif, hors noyau

Les relations structurelles utilisées pour l’analyse des goulots sont explicites :

$$K0 = K1 + K4 + K5$$

$$K1 = K2 + K3 + \text{reste}(K1)$$

Les blocs $P0$, $P1$ et $P2$ sont des coûts *one-shot* d’initialisation et ne doivent pas être mélangés aux métriques par itération. Les blocs $R0$ (rendu SDL) et $E0$ (gestion des événements SDL) sont mesurés séparément et traités comme non parallélisables dans l’analyse du noyau de calcul.

La stratégie instrumentale côté C++ s’appuie sur `timing_profile.hpp` et `timing_profile.cpp`. Le fichier `timing_profile.cpp` instancie l’état global de profilage (`g_timing_profile_totals`, `g_timing_profile_enabled`), tandis que `timing_profile.hpp` fournit l’horloge monotone (`profile_now_ns`) et les accumulateurs fins (`profile_add_k2`, `profile_add_k3`). L’accumulation des durées est effectuée bloc par bloc dans la boucle benchmark du simulateur, ce qui permet de séparer coûts d’initialisation, coûts noyau et coûts hors noyau.

L’automatisation est assurée par `scripts/measure_temps.sh`. Le pipeline suit une séquence reproductible : compilation release, puis 5 répétitions d’exécution avec 1200 itérations par répétition, 200 itérations de *warm-up* exclues et 1000 itérations effectivement mesurées, en configuration `OMP_NUM_THREADS=1`. Le script consolide les artefacts méthodologiques (`raw_measurements.csv`, `init_times.csv`, `table_a.csv`, `table_b.csv`, `amdahl_prediction.csv`, `summary.md`) afin de préparer la lecture des goulots. Les artefacts générés sont :

- `raw_measurements.csv` : mesures brutes par répétition (compteurs en ns et valeurs dérivées par itération).
- `init_times.csv` : synthèse des coûts *one-shot* d’initialisation ($P0$, $P1$, $P2$).
- `table_a.csv` : métriques des blocs de profilage (moyenne, dispersion et part relative du noyau).
- `table_b.csv` : indicateurs globaux par répétition (T_{iter} , T_{kernel} , T_{render} , débit).
- `amdahl_prediction.csv` : valeurs théoriques de speedup issues du modèle d’Amdahl à partir de la fraction parallélisable estimée.
- `summary.md` : synthèse lisible du protocole et des tableaux générés.

La méthode de lecture priorise les métriques `ms/iter` et % noyau pour identifier les *hotspots* dominants. Les conclusions doivent d’abord cibler les blocs du noyau ($K0$ – $K5$), puis isoler l’impact des composants hors noyau ($R0$, $E0$) pour éviter les biais d’interprétation.

2.3 Résultats

Les résultats présentés ci-dessous proviennent de la campagne de mesure :

`results/test_time_measurements/20260225_111731/summary.md`. La configuration utilisée est la suivante : 5 répétitions, 1200 itérations par répétition, 200 itérations de *warm-up*, 1000 itérations mesurées, et exécution avec `OMP_NUM_THREADS=1`.

TABLE 2 – Temps one-shot d’initialisation

Partie	ms moyen	écart-type
P0	41.581492	0.852773
P1	0.891350	0.031879
P2	0.076836	0.010131

TABLE 3 – Décomposition des coûts (profilage de base)

Partie	ms/iter moyen	écart-type	% kernel
K1	3.719856	0.005782	93.4676
K2	2.812386	0.005215	70.6659
K3	0.580826	0.002620	14.5942
K4	0.255192	0.004034	6.4121
K5	0.004597	0.000359	0.1155
R0	0.948657	0.011420	n/a
E0	0.001842	0.000113	n/a

TABLE 4 – Indicateurs globaux par répétition

Répétition	T_iter_total (ms)	T_kernel (ms)	T_render (ms)	fourmis/s
1	4.951205	3.989457	0.959952	1253303.431806
2	4.951423	3.991688	0.957977	1252602.984471
3	4.915457	3.972775	0.940620	1258566.168530
4	4.934418	3.978167	0.954489	1256860.215796
5	4.899159	3.967081	0.930245	1260372.560533
Moyenne	4.930332	3.979834	0.948657	1256341.072227
Écart-type	0.020446	0.009470	0.011420	2989.114282

Fraction parallélisable candidate :

$$F = (K1 + K4)/K0 = 0.998797$$

TABLE 5 – Prédiction théorique selon Amdahl

Threads	S(p)
2	1.997597
4	3.985616
8	7.933195
16	15.716398

2.3.1 Analyse des Résultats

Cette sous-section interprète les mesures présentées dans les tableaux 2, 3, 4 et 5, ainsi que les indicateurs F et $S(p)$. L’objectif est d’identifier les priorités de parallélisation à partir des données observées, en séparant clairement ce qui relève des coûts d’initialisation, du noyau de simulation et des bornes théoriques de gain.

1. **Coûts d’initialisation ($P0$ – $P2$).** Dans le tableau 2, $P0$ domine très largement la phase d’initialisation (41.581492 ms), alors que $P1$ (0.891350 ms) et $P2$ (0.076836 ms) restent faibles. Cette hiérarchie est cohérente avec le fait que la génération fractale du terrain est la tâche la plus lourde au démarrage. En revanche, comme $P0$ – $P2$ sont des coûts *one-shot* hors boucle itérative, ils ne constituent pas la priorité principale pour améliorer le temps de simulation par itération.
2. **Priorité de parallélisation au niveau noyau.** D’après le tableau 3, $K1$ représente 93.4676% du temps noyau, ce qui en fait la cible prioritaire. Le raisonnement est direct : réduire $K1$ a

l'impact potentiel maximal sur $K0$, avant $K4$ (évaporation) et $K5$ (mise à jour de la carte de phéromones), dont les contributions relatives sont nettement plus faibles.

3. **Décomposition interne de $K1$: $K2$ avant $K3$.** Toujours dans le tableau 3, $K2$ pèse 70.6659% du noyau contre 14.5942% pour $K3$. L'ordre de priorité est donc de paralléliser d'abord $K2$, puis $K3$. Cela confirme que la logique de déplacement/décision des fourmis est plus coûteuse que le marquage des phéromones.
4. **Stabilité expérimentale inter-répétitions.** Le tableau 4 montre une faible dispersion entre les 5 répétitions ($T_{\text{iter_total}}$, T_{kernel} , T_{render} , fourmis/s), ce qui indique une bonne stabilité de la campagne de mesure. Cette stabilité provient du protocole (phase de *warm-up* exclue et répétitions multiples), et non d'un effet spécifique "à partir de la cinquième itération".
5. **Interprétation de la fraction parallélisable F .** La valeur $F = 0.998797$, proche de 1, n'est pas anormale dans ce cadre : la définition $F = (K1 + K4)/K0$ inclut les blocs dominants du noyau et exclut les parties hors noyau ($R0$, $E0$). Cette valeur doit toutefois rester interprétée comme une fraction candidate théorique pour le noyau, et non comme une garantie de speedup réel, car les surcoûts de synchronisation, d'accès mémoire et d'équilibrage peuvent réduire le gain observé.
6. **Lecture correcte de la prédiction d'Amdahl.** Le tableau 5 montre un potentiel de gain important avec l'augmentation du nombre de threads. En particulier, $S(16) = 15.716398$ correspond à un facteur multiplicatif d'environ $\times 15.7$, et non à 15%. Cette tendance soutient clairement la stratégie de parallélisation ; la suite du projet doit donc viser d'abord $K1$, puis $K4$, pour se rapprocher de ce plafond théorique.

En synthèse, les deux *hotspots* centraux sont $K1$ au niveau global et $K2$ à l'intérieur de $K1$. Les mesures actuelles établissent une base robuste pour prioriser les optimisations, mais l'écart entre potentiel théorique et performance finale devra être vérifié expérimentalement après implémentation de la parallélisation. La feuille de route est donc cohérente : concentrer l'effort d'abord sur les blocs dominants du noyau afin de maximiser le retour de chaque évolution de code.

3 Vectorisation

3.1 Stratégie de Vectorisation

3.1.1 Motivation

Le point 5.1 de l'énoncé définit la *vectorisation* comme un changement de représentation des agents : au lieu de manipuler une liste d'objets « fourmi », on stocke séparément les attributs (coordonnées, état, graine de générateur pseudo-aléatoire, etc.) dans des tableaux. Une fourmi est alors repérée uniquement par son indice i dans ces tableaux.

Cette étape est motivée par l'analyse *baseline* : le bloc *K1* (avance de l'ensemble des fourmis) concentre l'essentiel du temps noyau. L'objectif principal est donc d'améliorer l'efficacité mémoire de cette boucle (localité, cache, bande passante) afin de réduire le coût par itération, tout en préparant la suite (parallélisation OpenMP au point 5.2 de la consigne).

3.1.2 Du layout AoS au layout SoA

Dans la version *baseline*, la population est stockée sous la forme d'un `std::vector<ant>` (*Array of Structs*, AoS). Chaque instance `ant` regroupe plusieurs champs hétérogènes (position `SDL_Point`, état, graine RNG, etc.). Lors de l'itération sur toutes les fourmis, le noyau lit/écrit principalement la position, l'état et la graine, mais les lignes de cache chargées contiennent aussi des données non nécessaires au pas courant.

Nous introduisons donc un layout *Structure of Arrays* (SoA), plus favorable à la localité et à la vectorisation au sens « données » du sujet :

$$\text{AoS} : \{(x_i, y_i, s_i, g_i)\}_{i=0}^{N-1} \longrightarrow \text{SoA} : (x[], y[], s[], g[]).$$

Ainsi, les accès séquentiels `x[i]`, `y[i]`, `state[i]`, `seed[i]` deviennent contigus en mémoire, ce qui augmente la proportion de données utiles par ligne de cache et facilite les boucles par indice.

3.1.3 Maintien du baseline : un binaire, deux layouts

Afin de conserver une référence expérimentale stable, le chemin AoS est maintenu et reste le mode d'exécution par défaut. Le programme expose un paramètre de ligne de commande permettant de choisir la représentation :

- `-layout aos` : exécution *baseline* (`std::vector<ant>`);
- `-layout soa` : exécution vectorisée (`AntsSoA`).

Ce choix « 1 binaire, 2 layouts » évite de dupliquer le projet en deux répertoires (et donc de diverger au fil des modifications) et garantit une comparaison directe AoS vs SoA à paramètres identiques.

3.1.4 Choix d'implémentation : types compacts et seed par fourmi

En SoA, nous stockons explicitement les attributs minimaux requis par la dynamique :

- `x[i]`, `y[i]` en `int32_t` (coordonnées);
- `state[i]` en `uint8_t` (0 = *unloaded*, 1 = *loaded*);
- `seed[i]` en `uint32_t` (état RNG local).

Le choix de types compacts réduit l’empreinte mémoire par agent et améliore l’utilisation du cache, ce qui est particulièrement pertinent puisque la boucle $K1$ parcourt plusieurs milliers de fourmis.

Un point important pour la reproductibilité est la gestion de la graine RNG par fourmi. Dans le code AoS *baseline*, le champ `m_seed` est utilisé dans `ant::advance (src/ant.cpp)` mais n’était pas initialisé dans le constructeur (`src/ant.hpp`), ce qui induit un *comportement indéfini* et rend le baseline instable. La vectorisation impose de rendre cette donnée explicite : chaque fourmi possède sa graine `seed[i]` et les fonctions de génération aléatoire la mettent à jour par référence. Pour garder une comparabilité maximale, la formule du générateur est conservée ; seule l’initialisation devient correcte et déterministe.

3.1.5 Application dans le noyau : `advance_time_soa`

La version SoA introduit un noyau `advance_time_soa` (analogue à `advance_time` dans `src/ant_simu.cpp`) :

1. boucle sur $i \in [0, N)$: avance de la fourmi i en manipulant `x[i]`, `y[i]`, `state[i]`, `seed[i]` ;
2. évaporation globale des phéromones (`do_evaporation`) ;
3. validation/swap de la carte de phéromones (`update`).

La logique par fourmi reste volontairement identique à la version AoS (mêmes choix de voisin, mêmes branches, même condition de mouvement `consumed_time < 1`) afin de comparer les performances en isolant l’effet du layout mémoire.

3.1.6 Découplage de `SDL_Point` hors hot loop et adaptation du rendu

Le type `SDL_Point` est pratique pour le rendu mais il couple le noyau de calcul à SDL. Dans la stratégie SoA, le noyau manipule des scalaires `int32_t` ; la conversion vers un point SDL (si nécessaire) est repoussée au moment du rendu. Dans le même esprit, une surcharge `mark_pheronome(x,y)` côté `pheronome` permet de marquer les phéromones sans construire un `SDL_Point` à chaque pas.

Enfin, le `Renderer` est adapté pour accepter soit une population AoS (`std::vector<ant>`), soit une population SoA (`AntsSoA`). Le rendu itère alors :

- en AoS : sur les objets et `ant.get_position()` ;
- en SoA : sur l’indice i et les tableaux `x[i]`/`y[i]`.

3.1.7 Validation et protocole de comparaison

La validation repose sur la compilation en mode *release* puis l’exécution du mode `-benchmark` en forçant successivement `-layout aos` puis `-layout soa`. Le format des métriques `METRIC ...` est conservé afin de réutiliser le pipeline de consolidation existant (`scripts/measure_temps.sh`). La comparaison se focalise sur $K1$ (et donc $K0$), puisque la vectorisation vise prioritairement le noyau d’avance des fourmis.